Nomad

A Functional Language with Borrow Checking

Walid Menou

Abstract

This project implements an interpreter for a functional programming language extended with mutability and borrow checking, investigating affine type systems for resource management. Borrow checking, famously used in Rust, provides strong compile-time guarantees about how values are used and mutated without relying on a garbage collector. A second objective is to explore expressiveness in language design by implementing features such as list comprehensions, a pipe operator, algebraic data types, pattern matching, and Hindley-Milner type inference. The developer should not have to manually annotate types or write redundant code, and should be able to express ideas concisely while still benefiting from strong compile-time guarantees. This report defines the language as it stands. It gives the grammar, the typing rules and the evaluation rules for every construct the interpreter supports, and it records the work still to come.

Contents

1 Introduction

Nomad is a small, strict, statically typed functional language in the ML family. A program is a list of top-level bindings and expressions. The interpreter reads the whole program, gives every expression a type, and only then runs it. No Nomad program carries a type annotation, because the checker works out every type on its own.

The aim is a language that solves programming challenges quickly. Someone working through a contest problem or an exercise set wants to write the idea down and get an answer. That person should not have to declare types twice, wrap input in a channel object, or spell out a fold that one line of set-builder notation would say. Nomad tries to keep the guarantees a strong type system gives while asking for none of the paperwork.

This report documents the language as the interpreter runs it today. Every example below was run, and the output shown beside it is what Nomad printed. The parser is a library of combinators in the style Hutton and Meijer set out [1].

Borrow checking is the next piece of work and the reason the project exists. It is not implemented yet. Section 6 lists what comes after it.

1.1 A first program

The program below sorts a list. It uses most of what the language offers: recursive bindings, functions of several arguments, list patterns and comprehensions.

```
let rec quicksort lst =
  match lst with
  [] -> []
  | x :: xs ->
      quicksort [y | y <- xs, y <= x] @ [x] @ quicksort [y | y <- xs, y > x]

let result = quicksort [5; 2; 9; 1; 5; 6]
```

Nothing in it names a type, and the checker still rejects it if any part does not fit. The two comprehensions are doing the work that two auxiliary filtering functions would otherwise have to do.

1.2 Running a program

The interpreter takes a file. Source files use the extension `.nd`.

```
$ nomad sort.nd
<fun> : int list -> int list
[1; 2; 5; 5; 6; 9]
```

Every statement prints the value it produced, binding forms included, which is why the two lines above correspond to the two bindings in the program. Values print as the source would write them, with strings in quotes and lists in brackets. A function has no useful printed form, so it shows as `<fun>` followed by its type.

Run `nomad` with no file and it reads statements one at a time instead. The rest of this report uses that prompt to show what an expression produces.

```
$ nomad
Nomad REPL (Type 'exit' or Ctrl+D to quit)
>> let xs = [1; 2; 3]
[1; 2; 3]
>> [x * x | x <- xs]
[1; 4; 9]
```

2 Syntax

2.1 Grammar

A program is a sequence of statements. Each statement either binds a name or evaluates an expression. A binding may take parameters, which is shorthand for a function that returns a function.

$$
\begin{array}{rcl}
\mathit{prog} & ::= & \mathit{stmt}^* \\
\mathit{stmt} & ::= & \textbf{let}\ x\ y_1 \ldots y_k = e \quad | \quad \textbf{let rec}\ f\ y_1 \ldots y_k = e \quad | \quad e \\
e & ::= & \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 \\
 & | & \textbf{fun}\ y_1 \ldots y_k \rightarrow e \\
 & | & \textbf{let}\ x\ y_1 \ldots y_k = e_1\ \textbf{in}\ e_2 \quad | \quad \textbf{let rec}\ f\ y_1 \ldots y_k = e_1\ \textbf{in}\ e_2 \\
 & | & \textbf{match}\ e\ \textbf{with}\ p_1 \rightarrow e_1 \mid \cdots \mid p_n \rightarrow e_n \\
 & | & e_1 \,\|\, e_2 \quad | \quad e_1 \,\&\&\, e_2 \quad | \quad e_1\ \mathit{relop}\ e_2 \\
 & | & e_1 \,|\!> e_2 \quad | \quad e_1 \,@\, e_2 \quad | \quad e_1 :: e_2 \\
 & | & e_1\ \mathit{addop}\ e_2 \quad | \quad e_1\ \mathit{mulop}\ e_2 \quad | \quad -e \quad | \quad e_1\ e_2 \\
 & | & x \quad | \quad n \quad | \quad b \quad | \quad s \quad | \quad () \quad | \quad (\,e\,) \quad | \quad (\,e_1, \ldots, e_n\,) \\
 & | & [\,e_1; \ldots; e_n\,] \quad | \quad [\,e_1 \,..\, e_2\,] \quad | \quad [\,e \mid q_1, \ldots, q_k\,] \\
q & ::= & p \leftarrow e \quad | \quad e \quad | \quad \textbf{let}\ x\ y_1 \ldots y_k = e \\
p & ::= & \_ \quad | \quad x \quad | \quad n \quad | \quad b \quad | \quad s \quad | \quad [\,] \quad | \quad p_1 :: p_2 \quad | \quad (\,p_1, \ldots, p_n\,) \\
\mathit{relop} & ::= & < \mid <= \mid > \mid >= \mid = \mid <> \\
\mathit{addop} & ::= & + \mid - \qquad \mathit{mulop} ::= * \mid / \mid \% \qquad b ::= \textbf{true} \mid \textbf{false}
\end{array}
$$

2.2 Operators

Types are written the way the interpreter prints them. `int`, `bool`, `string` and `unit` are the base types, `t list` is a list, `t1 -> t2` is a function, `(t1 * t2)` is a pair, and `'a` stands for a type that can be anything at all. Types never appear in Nomad source.

Table 1 gives every operator, what it accepts and what it produces.

Level	Operators	Type	Note
1 (loosest)	`||`	`bool -> bool -> bool`	left, short circuits
2	`&&`	`bool -> bool -> bool`	left, short circuits
3	`< <= > >=`	`int -> int -> bool`	chained
3	`= <>`	`'a -> 'a -> bool`	chained
4	`|>`	`'a -> ('a -> 'b) -> 'b`	left, pipe
5	`@`	`'a list -> 'a list -> 'a list`	right, append
6	`::`	`'a -> 'a list -> 'a list`	right, cons
7	`+ -`	`int -> int -> int`	left
8	`* / %`	`int -> int -> int`	left
9	`-` (prefix)	`int -> int`	negation
10 (tightest)	application	`('a -> 'b) -> 'a -> 'b`	left

Table 1: Operators, loosest first.

Both operands are evaluated, left one first, except for `&&` and `||`, which look at the right operand only when the left has not already settled the answer.

The four keyword forms `if`, `fun`, `let` and `match` run as far to the right as they can, which the table cannot show. In `if c then a else b + 1` the `else` branch is `b + 1`, and parentheses cut a form short.

2.3 Layout, comments and lexical rules

A statement ends where the next line begins in the first column. Whitespace inside a statement crosses a newline only when the line that follows is indented, so a continuation line must be indented and a top-level binding must not be. Two semicolons end a statement outright, which is the way to put more than one on a line, and they are optional everywhere else.

```
let a = 1 -- one statement
let b = -- another, continued below
  a + 1
let c = 2 ;; let d = 3 -- two on one line
```

A comment runs from `-` to the end of the line and counts as whitespace. Blank lines and comment lines are looked past when deciding whether the next line is indented, so a comment in the first column does not end the statement above it.

An identifier starts with a letter and continues with letters, digits and underscores. The words `let`, `in`, `fun`, `if`, `then`, `else`, `match` and `with` are reserved.

An integer literal carries no sign, since a leading minus is the prefix operator of level 9. This makes spacing matter in one place, and the rule is the one Haskell uses: `1 - 2` and `1 -2` are both subtraction, `1 -2` is `1` followed by a comment, and a negative right operand needs a space, as in `1 - -2`. A pattern keeps the signed literal, since a sign in a pattern cannot be an operator.

A string literal is any run of characters between two double quotes. There are no escape sequences.

3 The language

3.1 Literals

There are four kinds of literal.

```
>> 42
42
>> true
true
>> "hello"
"hello"
>> ()
()
```

Integers are machine integers, 63 bits wide on a 64-bit host, and they wrap on overflow. The unit value `()` is the only value of type `unit` and stands for a result that carries no information.

3.2 Variables and bindings

A `let` names a value. At the top level it stays in scope for the rest of the program, and with `in` it stays in scope for one expression. There is no way to declare a name without giving it a value.

```
>> let x = 21
21
>> x * 2
42
>> let y = let a = 2 in a * a
4
```

The bound expression runs once, whether or not the body uses it. A `let rec` puts the name in scope inside its own definition, which is the only way to write a loop, since the language has no loop construct.

```
>> let rec fact n = if n = 0 then 1 else n * fact (n - 1)
<fun> : int -> int
>> fact 5
120
```

Two functions that call each other cannot be written, because neither is in scope in the other's definition.

3.3 Arithmetic

The five arithmetic operators take integers and give integers. Division truncates towards zero and the remainder takes the sign of the left operand.

```
>> 7 / 2
```

```
3
>> -7 / 2
-3
>> -7 % 3
-1
>> 7 % -3
1
```

A zero right operand of `/` or `%` stops the program with `Evaluation Error: Division by zero`. Prefix minus is subtraction from zero.

3.4 Comparison and equality

The four relational operators compare integers. Equality and inequality compare two values of any one type, so both sides must agree but that type is otherwise free.

```
>> [1; 2] = [1; 2]
true
>> (1, true) = (1, true)
true
>> "a" <> "b"
true
```

Equality is structural, so two values of the same type are equal when they are built the same way. Reading two functions cannot decide whether they agree, so comparing them fails at run time with `Evaluation Error: Cannot compare functions`, and a function nested inside a list is caught the same way. Making that a static error would need a notion of equality type, which the language does not have.

3.5 Chained comparison

Mathematics writes $0 \leq x < 5$ and means both parts at once. Most languages read that as two comparisons in a row and produce nonsense. Nomad reads it the way mathematics does.

```
>> let x = 3
3
>> 0 <= x < 5
true
>> 1 = 1 = 1
true
```

A run of relational operators expands into a conjunction in which each operand is compared with both of its neighbours, so `a < b < c` means `a < b && b < c`. The conjunctions group to the left, and a single comparison is the case that expands to itself. The parser performs the expansion. Equality sits on this level too, so `a = b = c` asks whether all three agree rather than comparing a boolean to `c`.

Each interior operand appears twice in the expansion. Nothing in Nomad has a side effect, so this

changes only how much work the program does, but a language with input or mutation would have to bind the operands first.

3.6 Conjunction and disjunction

Both connectives take booleans and give a boolean, and both look at the right operand only when the left has not already settled the answer. A guard can therefore protect the expression after it.

```
>> false && 1 / 0 = 0
false
>> true || 1 / 0 = 0
true
```

Neither divides by zero, because neither right operand runs. This is what makes `i < n && a i = 0` safe to write.

3.7 Conditional

A conditional picks one of two branches. It is an expression, so it always produces a value and the `else` branch is never optional. Both branches must have the same type, since the checker cannot know which one will run, and only the branch taken is evaluated.

```
>> if 3 > 2 then "yes" else "no"
"yes"
```

3.8 Functions and application

A function takes one argument. Writing several parameters is shorthand: `fun x y -> e` is `fun x -> fun y -> e`, and `let f x y = e` binds `f` to the same thing. Applying such a function to fewer arguments than it expects gives back a function that remembers the ones supplied so far. Parameters carry no annotation, and the checker works out their types from how the body uses them.

```
>> let add x y = x + y
<fun> : int -> int -> int
>> add 2 3
5
>> let inc = add 1
<fun> : int -> int
>> inc 10
11
```

A function evaluates to a closure at once and the body waits until the function is applied. The closure keeps the environment the function was written in, which is what makes the language lexically scoped. A free variable in the body means what it meant where the function appears in the source, not what it happens to mean where the function is called.

```
>> let n = 10
10
>> let f = fun x -> x + n
<fun> : int -> int
>> let n = 0
0
>> f 1
11
```

Application is call by value. The evaluator reduces the function, then the argument, and runs the body in the closure's environment extended with the argument. A recursive closure also puts its own name back in scope before running the body, which is what lets the function call itself without a mutable cell.

3.9 Lists, cons, append and ranges

The list is the only compound data structure the language has. A list literal writes its elements between brackets separated by semicolons, and every element must have the same type. Cons puts one value on the front of a list, append joins two lists, and a range writes a run of consecutive integers with both ends included.

```
>> 1 :: [2; 3]
[1; 2; 3]
>> [1; 2] @ [3] @ [4]
[1; 2; 3; 4]
>> [1..5]
[1; 2; 3; 4; 5]
>> [3..1]
[]
```

The empty list leaves its element type free, so it can serve as a list of anything. Cons and append both build a new list and leave their arguments alone. Append groups to the right, so `a @ b @ c` reads as `a @ (b @ c)` and each append walks only its own left operand rather than the result accumulated so far.

3.10 Tuples

A tuple groups a fixed number of values of any types at all. The parentheses are always required, which is what keeps this comma from clashing with the one separating comprehension qualifiers, and a single parenthesised expression is that expression rather than a tuple of one.

```
>> (1, "a", true)
(1, "a", true)
```

Width is part of the type, so a pattern of the wrong width does not check.

3.11 Patterns

A pattern describes the shape of a value and names the parts worth keeping. There are eight forms.

Pattern	Matches	Binds
`_`	anything	nothing
`x`	anything	`x`
`42`	that integer	nothing
`true`	that boolean	nothing
`"ok"`	that string	nothing
`[]`	the empty list	nothing
`p1 :: p2`	a list with at least one element	what `p1` and `p2` bind
`(p1, ..., pn)`	a tuple of that width	what each part binds

Table 2: Pattern forms.

A name may appear only once in a pattern, since there would be no reason to prefer one of the two values it caught. What a pattern binds is visible in the body it guards and nowhere else, and a pattern variable hides an outer variable of the same name.

3.12 Pattern matching

A `match` tries its cases from top to bottom and runs the first one that fits. It is how a program takes a list apart, and with the constant patterns it also serves as a multi-way conditional. Every case must produce the same type and every pattern must match the type of the scrutinee.

```
>> let rec length l = match l with [] -> 0 | _ :: t -> 1 + length t
<fun> : 'a list -> int
>> length [1; 2; 3]
3
>> let fizz n = match n % 3 with 0 -> "fizz" | _ -> "no"
<fun> : int -> string
>> fizz 9
"fizz"
>> let fst p = match p with (a, _) -> a
<fun> : ('a * 'b) -> 'a
```

The cases must also cover every value the scrutinee could take. A match that leaves one out is rejected before the program runs.

```
>> let head l = match l with x :: _ -> x
Type Error: This match is not exhaustive
```

The checker decides this with Maranget's usefulness algorithm [7]: the patterns are exhaustive exactly when adding a further wildcard case would be useless. The type says which sets of constructors are complete, so both booleans suffice for a boolean, `[]` together with `::` suffice for a list, and one tuple pattern of the right width suffices for a tuple, while integers and strings are too many to cover

without a wildcard. The condition matters because Nomad has no exceptions. Without it a well typed program could still stop dead, and that was the one way it could.

3.13 Comprehensions

A comprehension writes a list by saying what its elements look like rather than by folding over another list. This is set-builder notation, so the bar reads as *such that*.

```
>> [x * x | x <- [1..10]]
[1; 4; 9; 16; 25; 36; 49; 64; 81; 100]
>> [x | x <- [-2; 3; -4; 5], x > 0]
[3; 5]
>> [y | x <- [1; 2; 3], let y = x * 10, y > 10]
[20; 30]
>> [x * 10 + y | x <- [1; 2], y <- [3; 4]]
[13; 14; 23; 24]
>> [x | r <- [[1; 2]; [3]], x <- r]
[1; 2; 3]
>> [h | h :: t <- [[1; 2]; []; [3]]]
[1; 3]
```

There are three kinds of qualifier: a generator `p <- e` draws values from a list, a guard is any boolean expression, and a `let` names a value. They run left to right and nest, so the leftmost generator is the outer loop and a later qualifier sees every name an earlier one bound. A guard skips the qualifiers after it for the value in hand. A generator pattern may be refutable, and a value it does not match is skipped rather than being an error, which is the one place the exhaustiveness condition deliberately does not apply. That is what the last line above relies on to take the heads of the non-empty lists.

The language is strict, so the whole list is built before anything else happens. Comprehensions came to functional programming through NPL and then Miranda, and the translation Wadler gives [8] is the one every implementation still uses. Nomad does not desugar, because there is no standard library to desugar into and a construct of its own gives errors that point at what the programmer wrote.

3.14 The pipe operator

The pipe writes a call with the argument on the left, so a chain of steps reads in the order the data flows. Instead of `h (g (f x))`, write `x |> f |> g |> h`. It is sugar, rewritten by the parser into `e2 e1`, and it groups to the left.

```
>> let double x = x * 2
<fun> : int -> int
>> [1; 2; 3] |> length |> double
6
```

3.15 Statements and programs

A program is a list of statements, and each statement sees the bindings the earlier ones made. There is no separate entry point and no `main`. The checker runs over the whole program before the evaluator runs over any of it, so a type error in the last statement stops the first one from running. Every statement then prints the value it produced, as Section 1.2 shows.

4 Type inference

A Nomad program carries no annotations, so the checker finds every type by itself. It uses the standard method: give the unknowns names, collect the constraints the program forces on those names, and solve them by unification. Pierce gives the full treatment [2], and the Cornell textbook works the same algorithm through in OCaml [3].

A type variable stands for a type nobody has pinned down. The checker makes a fresh one whenever it meets an unknown, such as a function parameter or the element type of an empty list, and unification then takes two types and finds the most general way to make them equal, or fails. The algorithm is Robinson's [4]. A variable must not be made equal to a type that contains it, since that would describe a type with no finite form. The occurs check rejects it.

```
>> fun x -> x x
Type Error: In x x: Recursive types not supported
```

A binding does not get one type. It gets a *scheme*, which is a type together with the variables in it that were left free, and every use of the name takes a fresh copy of those. This is full Hindley-Milner inference [5, 6], and it is what lets one definition serve several types.

```
>> let id x = x
<fun> : 'a -> 'a
>> (id 1, id "a")
(1, "a")
```

Without it no standard library could exist, because a single `map` would work on lists of integers or lists of strings but never both in one program. A top-level binding and a comprehension `let` generalise the same way.

Function parameters and pattern variables do not generalise. Their types are fixed by the caller, not by the binding, so a parameter used at two types is rejected, as it should be.

```
>> let bad = fun f -> if f true then f 1 else 0
Type Error: In f 1: Type mismatch: bool and int
```

Nomad needs no value restriction yet, because nothing in the language mutates. Once references arrive, generalising the type of an expression that allocates one would be unsound, and the restriction becomes necessary.

5 Nomad and Java

Nomad exists to state the answer to a programming challenge in as few lines as it takes to think of it. Java is the language most students reach for on the same problems, so it makes a fair yardstick for that one goal. What follows compares only what Nomad already has. Input and a standard library are not written yet, so this is not a scoreboard, and Section 6 lists what is missing. Every pair below was run.

5.1 Comprehensions

Every Pythagorean triple with all three sides at most `n`. Three generators and a guard say the whole problem.

```
let triples n =
  [[a; b; c] | c <- [1..n], b <- [1..c], a <- [1..b], a * a + b * b = c * c]

let result = triples 20
```

Java has no comprehension, so the loops have to be written out, and the result needs a container chosen for it. This is the whole file, because Java also needs a class, a `main` and a call to print a list.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Triples {
    static List<int[]> triples(int n) {
        List<int[]> out = new ArrayList<>();
        for (int c = 1; c <= n; c++)
            for (int b = 1; b <= c; b++)
                for (int a = 1; a <= b; a++)
                    if (a * a + b * b == c * c)
                        out.add(new int[] { a, b, c });
        return out;
    }

    public static void main(String[] args) {
        for (int[] t : triples(20))
            System.out.print(Arrays.toString(t) + " ");
        System.out.println();
    }
}
```

Both print the same six triples. Nomad states the condition and says nothing about how to search, while the Java version fixes the loop order, the container and the printing before it can say what a triple is. The remaining comparisons show the method only, but the class and the `main` are still there.

5.2 Chained comparison

A bound on both sides is common enough in these problems to be worth a syntax. Nomad takes it from mathematics.

```
>> let x = 3
3
>> 0 <= x < 5
true
```

The same line does not compile in Java, because `0 <= x` is a boolean and nothing compares a boolean to `5`.

```
System.out.println(0 <= x < 5);
                           ^
error: bad operand types for binary operator '<'
  first type: boolean
  second type: int
```

Java needs `0 <= x && x < 5`, which names `x` twice. That is a small cost on its own and a real one inside a nested loop where the bound is an expression rather than a variable.

5.3 Pattern matching

Merging two sorted lists is four cases: either list empty, and then whichever head is smaller. A match states the four cases and takes the lists apart in the same breath.

```
let rec merge xs ys =
  match xs with
  [] -> ys
  | x :: xr ->
      (match ys with
      [] -> xs
      | y :: yr -> if x <= y then x :: merge xr ys else y :: merge xs yr)
```

Java has no way to name the head and the tail while testing the shape, so the test and the destructuring are separate steps, and building the result takes an explicit accumulator.

```
static List<Integer> merge(List<Integer> xs, List<Integer> ys) {
    if (xs.isEmpty()) return ys;
    if (ys.isEmpty()) return xs;
    int x = xs.get(0), y = ys.get(0);
    List<Integer> out = new ArrayList<>();
    if (x <= y) {
        out.add(x);
        out.addAll(merge(xs.subList(1, xs.size()), ys));
    } else {
        out.add(y);
        out.addAll(merge(xs, ys.subList(1, ys.size())));
    }
```

```
    return out;
}
```

Both return `[1; 2; 3; 4; 5; 6]` on `[1; 4; 6]` and `[2; 3; 5]`.

5.4 Functions without annotations

Primes by trial division, using a higher-order `any` and a lambda for the divisibility test.

```
let not b = if b then false else true

let rec any p l =
  match l with
  [] -> false
  | x :: xs -> p x || any p xs

let primes n = [x | x <- [2..n], not (any (fun d -> x % d = 0) [2..x - 1])]
```

Nomad works out that `any` is `('a -> bool) -> 'a list -> bool` without being told anything. Java can express the same function, but only once the programmer has written the type parameter, the argument types, the return type and an import for the function type itself.

```
static <T> boolean any(Predicate<T> p, List<T> l) {
    for (T x : l)
        if (p.test(x)) return true;
    return false;
}
```

The lambda costs more too. `x` has to be copied into a final local before it can be captured, and the range has to be built as a list, so the one Nomad line becomes four.

```
for (int x = 2; x <= n; x++) {
    final int y = x;
    List<Integer> ds = IntStream.rangeClosed(2, x - 1).boxed().toList();
    if (!any(d -> y % d == 0, ds)) out.add(x);
}
```

Both print the ten primes below 30.

6 Future work

Here is a non-exhaustive list of features I would like to implement beyond this course.

A conditional expression. `x > 0 ? 1 : -1`, sugar for `if`.

Parallel generators. `[x + y | x <- xs | y <- ys]`, walking two lists in step rather than taking their product.

Comprehensions as functions. `[x * 2 | x <- _]` as a function of one list, with the hole marked rather than guessed.

Collectors. `sum[x * x | x <- [1..n]]`, so the same notation can fold instead of always building a list, as SRFI 42 does [9].

A standard library. Every program currently writes its own `concat` and `length`.

Functional arrays. Constant-time access from something that still behaves as a value, so that an update neither copies nor lets an old name see the new contents. Borrow checking is what would make that provable.

Matrices. Two-dimensional structures on top of those arrays, with native operations. Exploring ideas such as those in [10] would be worthwhile here.

Algebraic data types. Without them a recursive structure such as a trie cannot be given a type at all.

Input and output. Reading input should take one line, the way `cin » x » y` does in C++.

References

[1] Graham Hutton and Erik Meijer. *Monadic Parser Combinators*. Technical Report NOTTCS-TR-96-4, Department of Computer Science, University of Nottingham, 1996. https://people.cs.nott.ac.uk/pszgmh/monparsing.pdf

[2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[3] Michael R. Clarkson et al. *OCaml Programming: Correct + Efficient + Beautiful*. Cornell University, CS 3110. https://cs3110.github.io/textbook/cover.html

[4] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[5] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.

[6] Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM Symposium on Principles of Programming Languages*, pages 207–212, 1982.

[7] Luc Maranget. Warnings for pattern matching. *Journal of Functional Programming*, 17(3):387–421, 2007.

[8] Philip Wadler. List comprehensions. Chapter 7 of Simon L. Peyton Jones, *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[9] Sebastian Egner. *SRFI 42: Eager Comprehensions*. Scheme Requests for Implementation, 2003. https://srfi.schemers.org/srfi-42/srfi-42.html

[10] Jacob Errington. *Grids without indices*. 2025. https://jerrington.me/posts/2025-12-23-grids-without-indices.html